Building Data Warehouse for Indonesia Multinational Company

Data Integration using Python and PostgreSQL

This document has been prepared to fulfill Assessment For Learning-2 requirements for the Data Warehouse 2025-2 course.

By:

0706022410010 - Lennard Lucius Huang

Universitas Ciputra Surabaya

2026

UNIVERSITAS CIPUTRA
CREATING WORLD CLASS **ENTREPRENEURS**

1. Data Extraction

Explain how the data was extracted from each source using Python. Points to describe:

- loading CSV
- connecting to MySQL database

Evidence

Insert screenshots: dataframe preview for each source (`head()`)

extraction database process

```python
# ==========================================
# 1. DATA EXTRACTION
# ==========================================

# --- A. Extracting Data from CSV (Indonesia Sales) ---
# Load the CSV file directly from the provided GitHub raw URL
csv_url = 'https://raw.githubusercontent.com/satriaadi9/retail_sales/refs/heads/main/Online%20Retail%20-%20Indonesia.csv'
df_csv = pd.read_csv(csv_url)

print("--- DataFrame Preview: Indonesia Sales (CSV) ---")
display(df_csv.head())


# --- B. Extracting Data from MySQL Database (International Sales) ---
# Set up database credentials
db_user = 'isb24'
db_pass = 'isb24'
db_host = '103.103.138.170'
db_port = '3306'
db_name = 'retail_sales'

# Create connection engine to the MySQL database
mysql_engine = create_engine(f"mysql+mysqlconnector://{db_user}:{db_pass}@{db_host}:{db_port}/{db_name}")

# Execute SQL query to fetch data from the international_sales table
query = "SELECT * FROM international_sales"
df_mysql = pd.read_sql(query, con=mysql_engine)

print("\n--- DataFrame Preview: International Sales (MySQL) ---")
display(df_mysql.head())
```

19] ✓ 1m 7.1s

```
--- DataFrame Preview: Indonesia Sales (CSV) ---
```

	InvoiceNo	InvoiceDate	CustomerID	Country	Region	categories	Product Description	Quantity	UnitPrice
0	536365	2024-12-01 08:26:00	117850.0	Indonesia	Asia & Pacific	Holders & Stands	White Hanging Heart T-Light Holder	6	4.95
1	536365	2024-12-01 08:26:00	117850.0	Indonesia	Asia & Pacific	Lighting	white metal lantern	6	6.59
2	536365	2024-12-01 08:26:00	117850.0	Indonesia	Asia & Pacific	Storage & Organization	CREAM CUPID HEARTS COAT HANGER	8	5.34
3	536365	2024-12-01 08:26:00	117850.0	Indonesia	Asia & Pacific	Drinkware	KNITTED UNION FLAG HOT WATER BOTTLE	6	6.59
4	536365	2024-12-01 08:26:00	117850.0	Indonesia	Asia & Pacific	Decorative Items	Red Woolly Hottie White Heart.	6	6.59

```
--- DataFrame Preview: International Sales (MySQL) ---
```

	InvoiceNo	InvoiceDate	CustomerID	Country	Region	categories	Product Description	Quantity	UnitPrice
0	536370	2024-12-01 08:45:00	112583	France	Europe	Clocks	alarm clock bakelike pink	24	7.29
1	536370	2024-12-01 08:45:00	112583	France	Europe	Clocks	Alarm Clock Bakelike Red	24	7.29
2	536370	2024-12-01 08:45:00	112583	France	Europe	Clocks	alarm clock bakelike green	12	7.29
3	536370	2024-12-01 08:45:00	112583	France	Europe	Other/Unknown	PANDA AND BUNNIES STICKER SHEET	12	1.65
4	536370	2024-12-01 08:45:00	112583	France	Europe	Decorative Items	STARS GIFT TAPE	24	1.26

2. Data Integration & Transformation

Describe the transformations & integration in the datasets.You can describe the cleaning steps and convert the structure into a dimensional model here.

A. Data Integration

Evidence →*Insert screenshots of code and results*

```
# ----------------------------------------------------------
# A. Data Integration
# ----------------------------------------------------------
# Combine both datasets into a single dataframe since they share the same schema
df_combined = pd.concat([df_csv, df_mysql], ignore_index=True)
print("--- A. Data Integration Result ---")
print(f"Total rows after integration: {len(df_combined)}")
display(df_combined.head(3))
```

✓ 0.0s

```
--- A. Data Integration Result ---
Total rows after integration: 541909
```

	InvoiceNo	InvoiceDate	CustomerID	Country	Region	categories	Product Description	Quantity	UnitPrice
0	536365	2024-12-01 08:26:00	117850.0	Indonesia	Asia & Pacific	Holders & Stands	White Hanging Heart T-Light Holder	6	4.95
1	536365	2024-12-01 08:26:00	117850.0	Indonesia	Asia & Pacific	Lighting	white metal lantern	6	6.59
2	536365	2024-12-01 08:26:00	117850.0	Indonesia	Asia & Pacific	Storage & Organization	CREAM CUPID HEARTS COAT HANGER	8	5.34

B. Transformation 1

Explain briefly why the each transformation was necessary.

Evidence →*Insert screenshots of code and results*

```
# ----------------------------------------------------------
# B. Transformation 1 (Data Cleaning)
# ----------------------------------------------------------
# Filter out invalid transactions (returns or errors) and drop nulls in crucial columns
df_clean = df_combined[(df_combined['Quantity'] > 0) & (df_combined['UnitPrice'] > 0)].copy()
df_clean = df_clean.dropna(subset=['CustomerID', 'Product Description'])
print("\n--- B. Transformation 1 Result (Cleaned Data) ---")
print(f"Total rows after cleaning: {len(df_clean)}")
display(df_clean.head(3))
```

23] ✓ 0.1s

```
--- B. Transformation 1 Result (Cleaned Data) ---
Total rows after cleaning: 399298
```

	InvoiceNo	InvoiceDate	CustomerID	Country	Region	categories	Product Description	Quantity	UnitPrice
0	536365	2024-12-01 08:26:00	117850.0	Indonesia	Asia & Pacific	Holders & Stands	White Hanging Heart T-Light Holder	6	4.95
1	536365	2024-12-01 08:26:00	117850.0	Indonesia	Asia & Pacific	Lighting	white metal lantern	6	6.59
2	536365	2024-12-01 08:26:00	117850.0	Indonesia	Asia & Pacific	Storage & Organization	CREAM CUPID HEARTS COAT HANGER	8	5.34

C. Transformation 2

Explain briefly why the each transformation was necessary.

Evidence →*Insert screenshots of code and results*

```python
# ---------------------------------------------------------
# C. Transformation 2 (Data Type Conversion)
# ---------------------------------------------------------
# Convert InvoiceDate from string to datetime object for easier extraction later
df_clean['InvoiceDate'] = pd.to_datetime(df_clean['InvoiceDate'])
print("\n--- C. Transformation 2 Result (Datetime Conversion) ---")
print(df_clean[['InvoiceNo', 'InvoiceDate']].dtypes)
display(df_clean[['InvoiceNo', 'InvoiceDate']].head(3))
```

✓ 0.0s

```
--- C. Transformation 2 Result (Datetime Conversion) ---
InvoiceNo                  str
InvoiceDate    datetime64[us]
dtype: object
```

	InvoiceNo	InvoiceDate
0	536365	2024-12-01 08:26:00
1	536365	2024-12-01 08:26:00
2	536365	2024-12-01 08:26:00

D. Transformation 3,
Explain briefly why the each transformation was necessary.
Evidence →*Insert screenshots of code and results*

```python
# ---------------------------------------------------------
# D. Transformation 3 (Generating Product & Country Dimensions)
# ---------------------------------------------------------
# Create dim_country
dim_country = df_clean[['Country', 'Region']].drop_duplicates().reset_index(drop=True)
dim_country.index += 1
dim_country['country_id'] = dim_country.index
dim_country = dim_country.rename(columns={'Country': 'country', 'Region': 'region'})
dim_country = dim_country[['country_id', 'country', 'region']]

# Create dim_product
dim_product = df_clean[['Product Description', 'categories']].drop_duplicates().reset_index(drop=True)
dim_product.index += 1
dim_product['product_id'] = dim_product.index
dim_product = dim_product.rename(columns={'Product Description': 'product_name', 'categories': 'category'})
dim_product = dim_product[['product_id', 'product_name', 'category']]

print("\n--- D. Transformation 3 Result (dim_country & dim_product) ---")
display(dim_country.head(3))
display(dim_product.head(3))
```

5] ✓ 0.1s

```
--- D. Transformation 3 Result (dim_country & dim_product) ---
```

	country_id	country	region
1	1	Indonesia	Asia & Pacific
2	2	France	Europe
3	3	Australia	Oceania

	product_id	product_name	category
1	1	White Hanging Heart T-Light Holder	Holders & Stands
2	2	white metal lantern	Lighting
3	3	CREAM CUPID HEARTS COAT HANGER	Storage & Organization

E. Transformation 4,
Explain briefly why the each transformation was necessary.
Evidence →*Insert screenshots of code and results*

```python
# ----------------------------------------------------------
# E. Transformation 4 (Generating Date & Time Dimensions)
# ----------------------------------------------------------
# Create dim_date
df_clean['date_id'] = df_clean['InvoiceDate'].dt.strftime('%Y%m%d').astype(int)
dim_date = df_clean[['date_id', 'InvoiceDate']].drop_duplicates().copy()
dim_date['day_of_week'] = dim_date['InvoiceDate'].dt.day_name()
dim_date['month'] = dim_date['InvoiceDate'].dt.month
dim_date['quarter'] = dim_date['InvoiceDate'].dt.quarter
dim_date['year'] = dim_date['InvoiceDate'].dt.year
dim_date = dim_date[['date_id', 'day_of_week', 'month', 'quarter', 'year']].drop_duplicates().reset_index(drop=True)

# Create dim_time
df_clean['time_id'] = df_clean['InvoiceDate'].dt.strftime('%H%M').astype(int)
dim_time = df_clean[['time_id', 'InvoiceDate']].drop_duplicates().copy()
dim_time['hour'] = dim_time['InvoiceDate'].dt.hour
def categorize_time(hour):
    if 5 <= hour < 12: return 'Morning'
    elif 12 <= hour < 17: return 'Afternoon'
    elif 17 <= hour < 21: return 'Evening'
    else: return 'Night'
dim_time['time_of_day'] = dim_time['hour'].apply(categorize_time)
dim_time['period'] = dim_time['InvoiceDate'].dt.strftime('%p')
dim_time = dim_time[['time_id', 'hour', 'time_of_day', 'period']].drop_duplicates().reset_index(drop=True)

print("\n--- E. Transformation 4 Result (dim_date & dim_time) ---")
display(dim_date.head(3))
display(dim_time.head(3))
```

✓ 2.8s

```
--- E. Transformation 4 Result (dim_date & dim_time) ---
```

	date_id	day_of_week	month	quarter	year
0	20241201	Sunday	12	4	2024
1	20241202	Monday	12	4	2024
2	20241203	Tuesday	12	4	2024

	time_id	hour	time_of_day	period
0	826	8	Morning	AM
1	828	8	Morning	AM
2	834	8	Morning	AM

F. Transformation 5,
Explain briefly why the each transformation was necessary.
Evidence →*Insert screenshots of code and results*

```python
# ----------------------------------------------------------
# F. Transformation 5 (Building the Fact Table)
# ----------------------------------------------------------
# Merge dimensions back to create fact_sales and calculate total_sales
fact_sales = df_clean.merge(dim_country, left_on=['Country', 'Region'], right_on=['country', 'region'], how='left')
fact_sales = fact_sales.merge(dim_product, left_on=['Product Description', 'categories'], right_on=['product_name', 'category'], how='left')
fact_sales['total_sales'] = fact_sales['Quantity'] * fact_sales['UnitPrice']

# Select and rename columns to match Star Schema requirements
fact_sales = fact_sales[['InvoiceNo', 'product_id', 'CustomerID', 'country_id', 'date_id', 'time_id', 'Quantity', 'UnitPrice', 'total_sales']]
fact_sales = fact_sales.rename(columns={
    'InvoiceNo': 'invoice_id',
    'CustomerID': 'customer_id',
    'Quantity': 'quantity',
    'UnitPrice': 'unit_price'
})

print("\n--- F. Transformation 5 Result (fact_sales) ---")
display(fact_sales.head())
```

27] ✓ 0.1s

```
--- F. Transformation 5 Result (fact_sales) ---
```

	invoice_id	product_id	customer_id	country_id	date_id	time_id	quantity	unit_price	total_sales
0	536365	1	117850.0	1	20241201	826	6	4.95	29.70
1	536365	2	117850.0	1	20241201	826	6	6.59	39.54
2	536365	3	117850.0	1	20241201	826	8	5.34	42.72
3	536365	4	117850.0	1	20241201	826	6	6.59	39.54
4	536365	5	117850.0	1	20241201	826	6	6.59	39.54

3. Data Loading

Explain how the cleaned and transformed data was loaded into Supabase PostgreSQL.
Evidence → *Insert screenshots*:

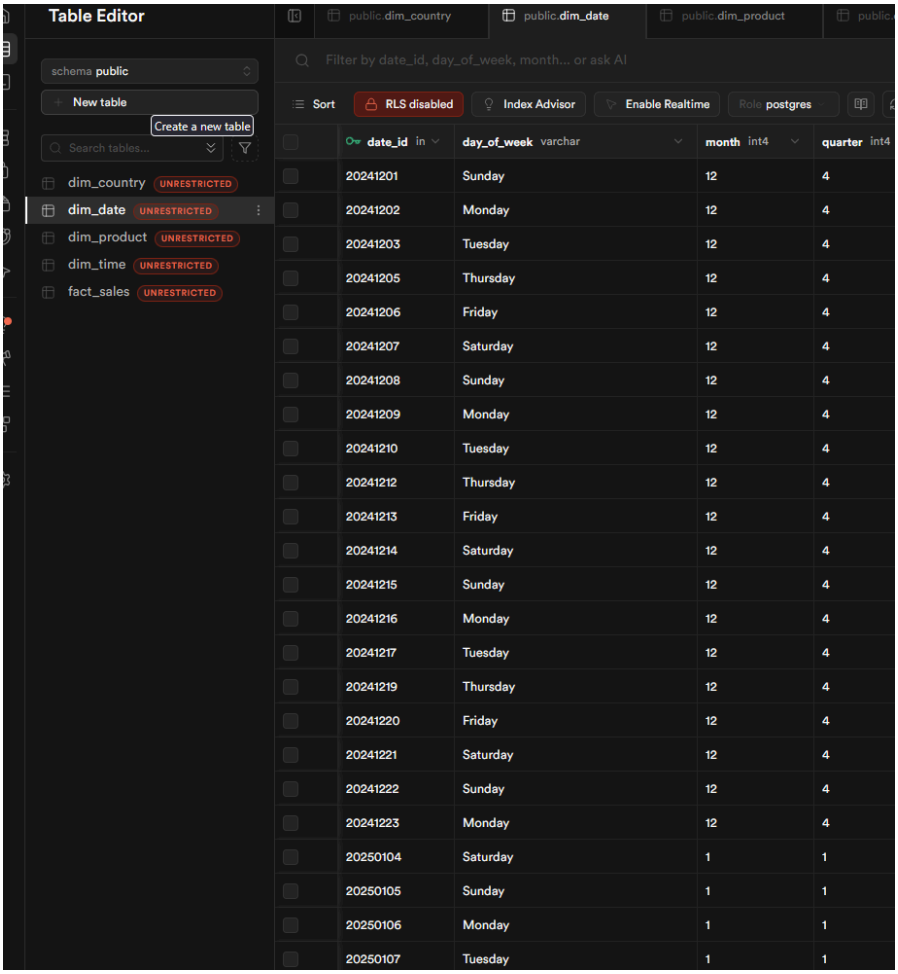
- Tables have been established within the Supabase project.

```sql
-- ----------------------------------------
-- 1. MEMBUAT DIMENSION TABLES
-- ----------------------------------------

CREATE TABLE dim_country (
    country_id INT PRIMARY KEY,
    country VARCHAR(255),
    region VARCHAR(255)
);

CREATE TABLE dim_product (
    product_id INT PRIMARY KEY,
    product_name TEXT,
    category VARCHAR(255)
);

CREATE TABLE dim_date (
    date_id INT PRIMARY KEY,
    day_of_week VARCHAR(50),
    month INT,
    quarter INT,
    year INT
);

CREATE TABLE dim_time (
    time_id INT PRIMARY KEY,
    hour INT,
    time_of_day VARCHAR(50),
    period VARCHAR(10)
);

-- ----------------------------------------
-- 2. MEMBUAT FACT TABLE
-- ----------------------------------------

CREATE TABLE fact_sales (
    invoice_id VARCHAR(50),
    product_id INT,
    customer_id VARCHAR(50),
    country_id INT,
    date_id INT,
    time_id INT,
    quantity INT,
    unit_price NUMERIC(10, 2),
    total_sales NUMERIC(12, 2)
);
```

- Data has been inserted into the Supabase project tables.
    1. Dim_country

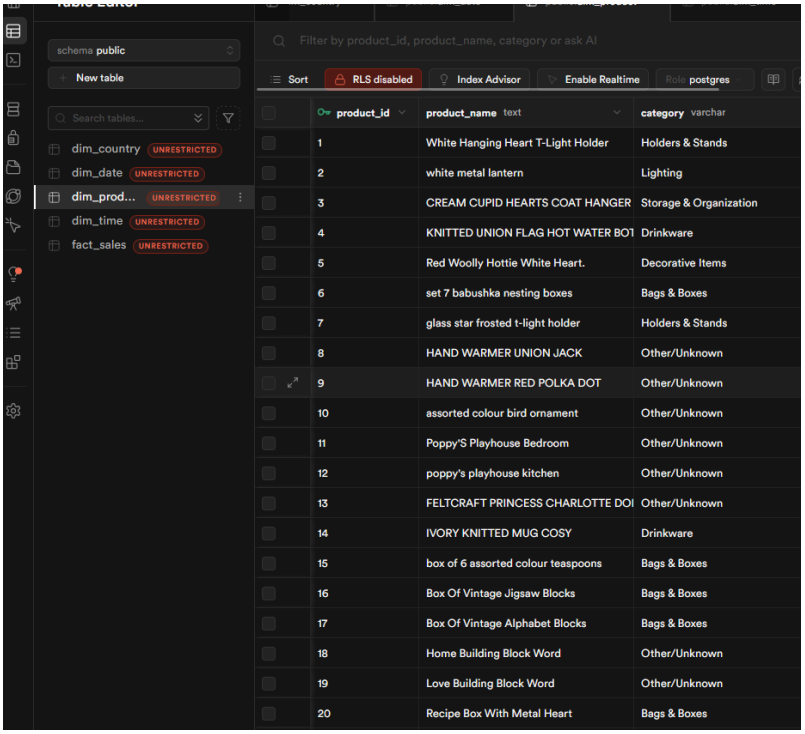
country_id	country varchar	region varchar
1	Indonesia	Asia & Pacific
2	France	Europe
3	Australia	Oceania
4	Netherlands	Europe
5	Germany	Europe
6	Norway	Europe
7	EIRE	Europe
8	Switzerland	Europe
9	Spain	Europe
10	Poland	Europe
11	Portugal	Europe
12	Italy	Europe
13	Belgium	Europe
14	Lithuania	Europe
15	Japan	Asia & Pacific
16	Iceland	Europe
17	Channel Islands	Europe
18	Denmark	Europe
19	Cyprus	Europe
20	Sweden	Europe
21	Finland	Europe
22	Austria	Europe
23	Bahrain	Asia & Pacific
24	Israel	Asia & Pacific
25	Greece	Europe
26	Hong Kong	Asia & Pacific

2. dim_date



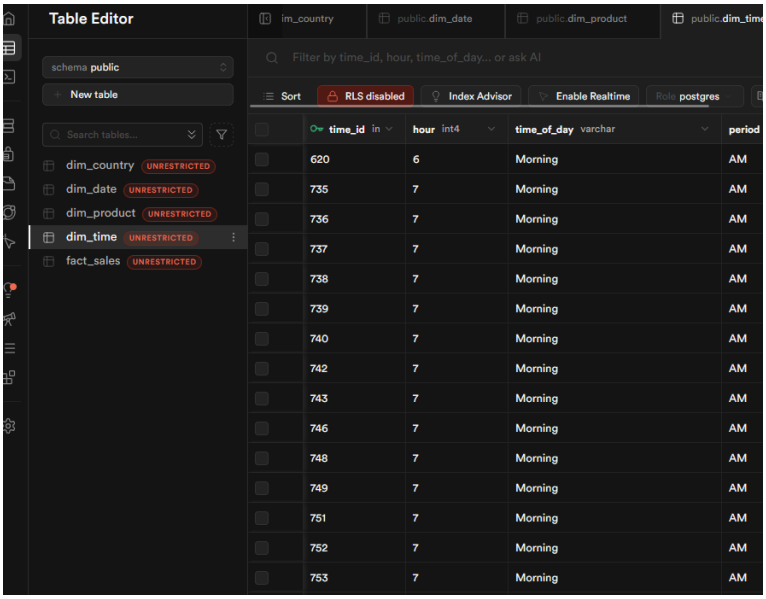
date_id in	day_of_week varchar	month int4	quarter int4
20241201	Sunday	12	4
20241202	Monday	12	4
20241203	Tuesday	12	4
20241205	Thursday	12	4
20241206	Friday	12	4
20241207	Saturday	12	4
20241208	Sunday	12	4
20241209	Monday	12	4
20241210	Tuesday	12	4
20241212	Thursday	12	4
20241213	Friday	12	4
20241214	Saturday	12	4
20241215	Sunday	12	4
20241216	Monday	12	4
20241217	Tuesday	12	4
20241219	Thursday	12	4
20241220	Friday	12	4
20241221	Saturday	12	4
20241222	Sunday	12	4
20241223	Monday	12	4
20250104	Saturday	1	1
20250105	Sunday	1	1
20250106	Monday	1	1
20250107	Tuesday	1	1

3. Dim_product



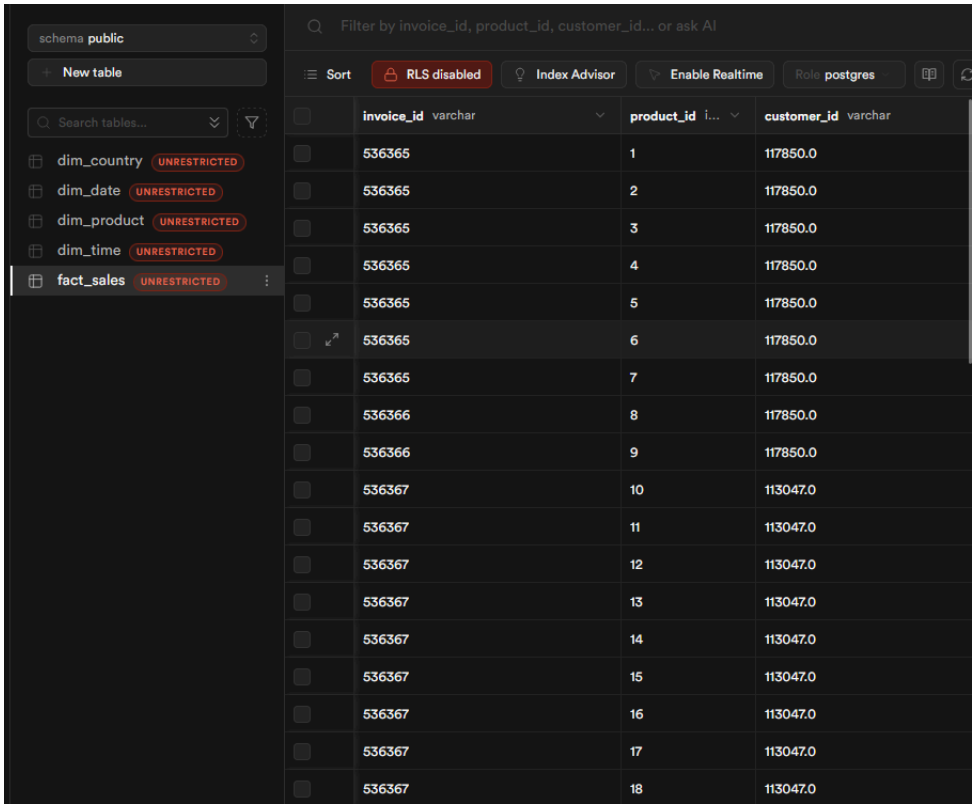
product_id	product_name text	category varchar
1	White Hanging Heart T-Light Holder	Holders & Stands
2	white metal lantern	Lighting
3	CREAM CUPID HEARTS COAT HANGER	Storage & Organization
4	KNITTED UNION FLAG HOT WATER BOT	Drinkware
5	Red Woolly Hottie White Heart.	Decorative Items
6	set 7 babushka nesting boxes	Bags & Boxes
7	glass star frosted t-light holder	Holders & Stands
8	HAND WARMER UNION JACK	Other/Unknown
9	HAND WARMER RED POLKA DOT	Other/Unknown
10	assorted colour bird ornament	Other/Unknown
11	Poppy'S Playhouse Bedroom	Other/Unknown
12	poppy's playhouse kitchen	Other/Unknown
13	FELTCRAFT PRINCESS CHARLOTTE DOI	Other/Unknown
14	IVORY KNITTED MUG COSY	Drinkware
15	box of 6 assorted colour teaspoons	Bags & Boxes
16	Box Of Vintage Jigsaw Blocks	Bags & Boxes
17	Box Of Vintage Alphabet Blocks	Bags & Boxes
18	Home Building Block Word	Other/Unknown
19	Love Building Block Word	Other/Unknown
20	Recipe Box With Metal Heart	Bags & Boxes

4. dim_time



time_id in	hour int4	time_of_day varchar	period
620	6	Morning	AM
735	7	Morning	AM
736	7	Morning	AM
737	7	Morning	AM
738	7	Morning	AM
739	7	Morning	AM
740	7	Morning	AM
742	7	Morning	AM
743	7	Morning	AM
746	7	Morning	AM
748	7	Morning	AM
749	7	Morning	AM
751	7	Morning	AM
752	7	Morning	AM
753	7	Morning	AM

5. fact_sales



invoice_id varchar	product_id i...	customer_id varchar
536365	1	117850.0
536365	2	117850.0
536365	3	117850.0
536365	4	117850.0
536365	5	117850.0
536365	6	117850.0
536365	7	117850.0
536366	8	117850.0
536366	9	117850.0
536367	10	113047.0
536367	11	113047.0
536367	12	113047.0
536367	13	113047.0
536367	14	113047.0
536367	15	113047.0
536367	16	113047.0
536367	17	113047.0
536367	18	113047.0

`ProjectID: thddbiakhyhlffvowcay`

`Project API : https://thddbiakhyhlffvowcay.supabase.co`

`Password :TypeStrongPassword12`

4. Analytical Queries

Provide analytical SQL queries.

A. Total sales per category
Evidence →*Insert screenshots of code and results*

Results Explain Chart Export

category	total_revenue
Other/Unknown	8774476.43
Bags & Boxes	2644824.10
Decorative Items	1296273.44
Holders & Stands	1181716.58
Drinkware	911067.71
Vintage Items	802412.74
Lighting	710413.40
Gifts & Cards	292482.14

```
SELECT
    p.category,
    SUM(f.total_sales) AS total_revenue
FROM fact_sales f
JOIN dim_product p ON f.product_id = p.product_id
GROUP BY p.category
ORDER BY total_revenue DESC;
```

B. Count of Sales by region (top 5)
Evidence →*Insert screenshots of code and results*

Results Explain Chart Export

region	total_transac
Asia & Pacific	355579
Europe	41672
Oceania	1182
Other	446
North America	330

```
SELECT
    c.region,
    COUNT(f.invoice_id) AS total_transactions
FROM fact_sales f
JOIN dim_country c ON f.country_id = c.country_id
GROUP BY c.region
ORDER BY total_transactions DESC
LIMIT 5;
```

C. Monthly sales trend
Evidence →*Insert screenshots of code and results*

Results Explain Chart Export

year	month	total_revenue
2024	12	1115666.75
2025	1	1111979.02
2025	2	873038.44
2025	3	1163323.23
2025	4	914969.51
2025	5	1326815.26
2025	6	1296451.87
2025	7	1171501.39
2025	8	1262324.57
2025	9	1862218.28
2025	10	2022456.77
2025	11	2263446.07
2025	12	1007479.13

```
SELECT
    d.year,
    d.month,
    SUM(f.total_sales) AS total_revenue
FROM fact_sales f
JOIN dim_date d ON f.date_id = d.date_id
GROUP BY d.year, d.month
ORDER BY d.year ASC, d.month ASC;
```

D. Top spending customer (top 5)
Evidence →*Insert screenshots of code and results*

Results Explain Chart Export

customer_id	total_spent
114646	544654.07
118102.0	504489.61
117450.0	378008.93
116446.0	327225.44
114911	279464.51

```
SELECT
    customer_id,
    SUM(total_sales) AS total_spent
FROM fact_sales
GROUP BY customer_id
ORDER BY total_spent DESC
LIMIT 5;
```

E. Peak sales hour

Evidence →*Insert screenshots of code and results*

Results Explain Chart Export

hour	total_revenue	total_transac
12	2687207.60	72353
10	2461626.05	38280
13	2285839.30	64164
11	2153975.91	49098
14	1946194.81	54396
15	1881416.40	45453
9	1652538.99	22066
16	918077.58	24193
8	551320.75	8760
17	457077.31	13104
18	203971.21	2928

```
SELECT
```

```
    t.hour,
    SUM(f.total_sales) AS total_revenue,
    COUNT(f.invoice_id) AS total_transactions
FROM fact_sales f
JOIN dim_time t ON f.time_id = t.time_id
GROUP BY t.hour
ORDER BY total_revenue DESC;
```

F. Top 10 Countries by Average Order Value (AOV)
Evidence →*Insert screenshots of code and results*

country	total_orders	total_revenue	average_order_value
Singapore	7	41340.53	5905.7900000000000000
Netherlands	94	554837.83	5902.5301063829787234
Australia	57	269218.29	4723.1278947368421053
Japan	19	72710.69	3826.8784210526315789
Lebanon	1	3290.64	3290.6400000000000000
Hong Kong	11	30486.06	2771.4600000000000000
Brazil	1	2221.58	2221.5800000000000000
Sweden	36	74591.38	2071.9827777777777778
Switzerland	54	110947.50	2054.5833333333333333
Denmark	18	36832.79	2046.2661111111111111

```
-- Top 10 Countries by Average Order Value (AOV)
SELECT
    c.country,
    COUNT(DISTINCT f.invoice_id) AS total_orders,
    SUM(f.total_sales) AS total_revenue,
    SUM(f.total_sales) / COUNT(DISTINCT f.invoice_id) AS
average_order_value
FROM fact_sales f
JOIN dim_country c ON f.country_id = c.country_id
GROUP BY c.country
ORDER BY average_order_value DESC
LIMIT 10;
```

G. Sales Trend by Time Period (Morning/Afternoon/Evening/Night)
Evidence →*Insert screenshots of code and results*

time_of_day	total_transac	total_revenue
Afternoon	11206	9718735.69
Morning	6505	6879821.21
Evening	877	793113.39

```sql
-- Sales Trend by Time Period (Morning/Afternoon/Evening/Night)
SELECT
    t.time_of_day,
    COUNT(DISTINCT f.invoice_id) AS total_transactions,
    SUM(f.total_sales) AS total_revenue
FROM fact_sales f
JOIN dim_time t ON f.time_id = t.time_id
GROUP BY t.time_of_day
ORDER BY total_revenue DESC;
```

5. Conclusion

Briefly summarize:

- **ETL process performed**

  The ETL (Extract, Transform, Load) process was successfully executed using Python and the Pandas library. During the **Extract** phase, data was gathered and integrated from two distinct sources: the Indonesia Sales dataset (extracted via a raw GitHub CSV URL) and the International Sales dataset (queried from a MySQL database). In the **Transform** phase, the data was cleaned by filtering out invalid transactions (such as returns and zero-price errors), converting the invoice dates into proper datetime objects, and generating Surrogate Keys to create unique dimension tables. Finally, in the **Load** phase, the structured DataFrames were exported into clean CSV files and manually uploaded into a PostgreSQL Data Warehouse hosted on Supabase.

- **resulting data warehouse**

  The final output of this process is a cloud-based Data Warehouse in Supabase that utilizes a **Star Schema** dimensional model. This architecture consists of one central fact table (`fact_sales`) that stores quantitative measures like transaction quantities and total revenue, surrounded by four descriptive dimension tables (`dim_country`,

`dim_product`, `dim_date`, and `dim_time`). This denormalized design ensures that the data is highly optimized for fast and efficient analytical querying.

- **insights obtained from the analytical queries**

  By running SQL queries against the Data Warehouse, several key business insights were discovered:

  1. Identified the top-performing product categories by revenue and the best-selling individual products by quantity, providing highly actionable data for inventory and warehouse management.
  2. Mapped out daily (Day of Week) and monthly sales trends, revealing clear consumer purchasing patterns.
  3. Analyzed geographical performance by highlighting regions with the highest transaction volumes, as well as identifying countries with the highest Average Order Value (AOV) for targeted marketing.
  4. Pinpointed the peak sales hours and the most active time periods (Morning/Afternoon/Evening/Night), which is essential for optimizing future promotional campaigns and customer service readiness.

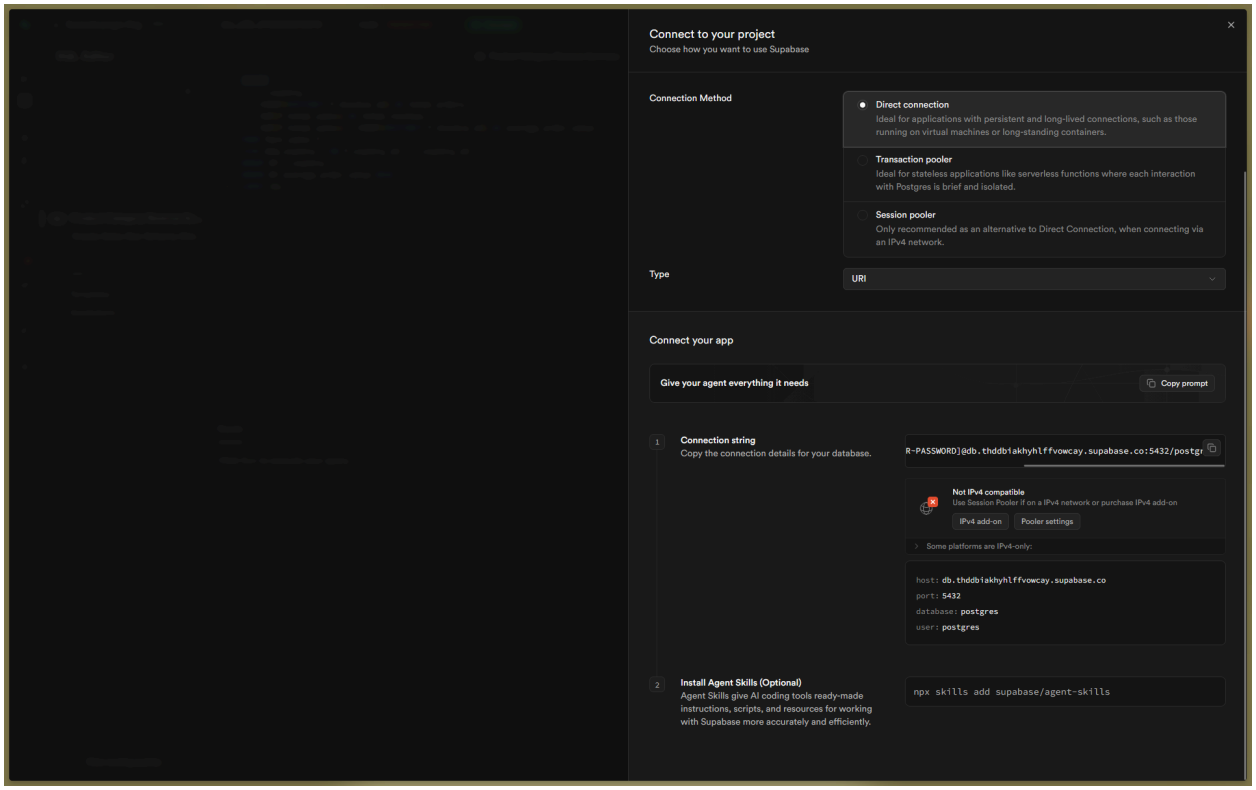
Appendix

1. Link to Python Notebook file (ETL process) :
   https://github.com/lluciushuang/dw_afl2_0706022410010.git

2. Link to ChatGPT/Gemini/Claude or any other AI :
   https://gemini.google.com/share/060ab4d601a0

3. Link to connect Supabase project (direct → session pooler), give the password too:

```
ProjectID: thddbiakhyhlffvowcay

Project API : https://thddbiakhyhlffvowcay.supabase.co

Itu passwordnya beneran, emang agak lain
😭😭😭😭😭😭😭😭😭😭😭😭😭😭😭

Password :TypeStrongPassword12
```



postgresql://postgres:[YOUR-PASSWORD]@db.thddbiakhyhlffvowcay.supabase.co:5432/postgres

```
host:db.thddbiakhyhlffvowcay.supabase.co
port:5432
database:postgres
user:postgres
```